EXPERIMENT NO. 2

Implement Data Types, Variables, Type Conversion and Operators

PART B – TO BE COMPLETED BY STUDENTS

Roll No. C-60

Name Avdhut Parish Madhavi

Class / Division SE-C

Batch C2

Date of Experiment _____

Date of Submission _____

Grade _____

B.1 Document Created by the Student

Modify the program using your own sample values and record the output.

Modified sample values:

- Pulse = 80
- Temperature = 98
- Weight = 70.0 kg
- Height = 1.75 m

Compilation command:

javac HealthMetrics.java

Execution command:

java HealthMetrics

B.2 Student Program / Output

Paste/write the final program and attach or reproduce the observed output below.

Java Code:

```
public class HealthMetrics {
   public static void main(String[] args) {
      int pulse = 80;
      int temperature = 98;
      double weight = 70.0;
      double height = 1.75;

      double bmi = weight / (height * height);
      int pulsePlus = pulse + 5;
      boolean normalPulse = pulse >= 60 && pulse <= 100;

      int flags = 40;
      int shifted = flags >>> 2;
```

```
        double integerToDouble = pulse;
        int doubleToInteger = (int) weight;

        System.out.println("BMI = " + bmi);
        System.out.println("Pulse + 5 = " + pulsePlus);
        System.out.println("Pulse in normal range = " + normalPulse);
        System.out.println("40 >>> 2 = " + shifted);
        System.out.println("Widening conversion = " + integerToDouble);
        System.out.println("Narrowing conversion = " + doubleToInteger);
    }
}
```

Output:

```
BMI = 22.857142857142858
Pulse + 5 = 85
Pulse in normal range = true
40 >>> 2 = 10
Widening conversion = 80.0
Narrowing conversion = 70
```

B.3 Observations and Learning

- What is the difference between widening and narrowing conversion?

  Widening conversion: Converts a smaller data type into a larger compatible data type. It is generally performed automatically. Example: int to double.
  Narrowing conversion: Converts a larger data type into a smaller data type. It generally requires explicit casting and may result in data loss. Example: double to int.

- How does >>> differ from >>?

  >> is the signed right-shift operator. It preserves the sign bit when shifting.
  >>> is the unsigned right-shift operator. It fills the leftmost positions with zeros.

- Which operator is appropriate for combining two Boolean conditions?

  The logical AND operator (&&) is used to combine two Boolean conditions when both conditions must be true.

Student observation / learning:

The program demonstrates Java variables, primitive data types, arithmetic calculations, relational and logical operators, bitwise shifting, and type conversion. Widening conversion automatically converts an integer into a double, while narrowing conversion uses explicit casting to convert a double into an integer.

B.4 Conclusion

Sample health values are used to demonstrate programming concepts; they are not medical diagnostic rules.

Student conclusion:

The experiment demonstrated the use of Java data types, variables, arithmetic and logical operators, unsigned right shift, and widening and narrowing conversions. The program executed successfully and displayed the calculated results.

B.5 Questions of Curiosity

1. What is the difference between primitive and reference data types?

Primitive data types: Store simple values directly. Examples include `int`, `char`, `double`, and `boolean`.

Reference data types: Store references to objects or arrays. Examples include `String`, arrays, and user-defined classes.

2. Why is explicit casting needed in narrowing conversion?
Explicit casting is needed because narrowing conversion may lose information or precision. Casting tells the compiler that the programmer intends to perform the conversion.
Example:

```
double value = 62.5;
int result = (int) value;
System.out.println(result);
```

3. How does the unsigned right-shift operator >>> work?

The >>> operator shifts the bits of a number to the right and fills the empty leftmost positions with zeros.

Example:

```
int flags = 40;
System.out.println(flags >>> 2);
```

Output:

10